

Show understanding of the relationship between assembly language and machine code

9618_w21_qp_11_sc_4.d

QUESTION

(d) Francis's team use language translators.

Complete the descriptions of language translators by writing the missing words.

..... are usually used when a high-level language program is complete. They translate all the code at the same time and then run the program.

They produce files that can be run without the source code.

..... translate one line of a high-level language program at a time, and then run that line of code. They are most useful while developing the programs because errors can be corrected and then the program continues from that line.

Assemblers are used to translate assembly code into

[4]

MARK SCHEME

4(d)	1 mark for each correctly completed term Compilers are usually used when a high-level language program is complete. They translate all the code at the same time and then run the program. They produce executable/.exe/object code files that can be run without the source code. Interpreters translate one line of a high-level language program at a time, and then run that line of code. They are most useful while developing the programs because errors can be corrected and then the program continues from that line. Assemblers are used to translate assembly code into binary/machine code .	4
------	--	----------

Describe the different stages of the assembly process for a two-pass assembler

9618_w23_qp_11_sc_8.a

QUESTION

- 8 (a) Identify the purpose of the first pass of a two-pass assembler.

.....
..... [1]

MARK SCHEME

8(a)	1 mark for: To create a symbol table	1
------	--	----------

9618_s23_qp_12_sc_3.a

QUESTION

- 3 A program is written in assembly language.

- (a) The program is converted into machine code by a two-pass assembler.

Draw **one or more** lines to identify the pass or passes in which each action takes place.

Action	Pass
generates object code	
reads the source code one line at a time	first
removes white space	
adds labels to the symbol table	second

[3]

MARK SCHEME

3(a)	1 mark for generates object code to second pass 1 mark for reads source code one line at a time to both boxes 1 mark for removes white space and adds labels to first pass <table><thead><tr><th>Action</th><th>Pass</th></tr></thead><tbody><tr><td>generates object code</td><td>first</td></tr><tr><td>reads the source code one line at a time</td><td>first</td></tr><tr><td>reads the source code one line at a time</td><td>second</td></tr><tr><td>removes white space</td><td>first</td></tr><tr><td>removes white space</td><td>second</td></tr><tr><td>adds labels to the symbol table</td><td>first</td></tr><tr><td>adds labels to the symbol table</td><td>second</td></tr></tbody></table>	Action	Pass	generates object code	first	reads the source code one line at a time	first	reads the source code one line at a time	second	removes white space	first	removes white space	second	adds labels to the symbol table	first	adds labels to the symbol table	second	3
Action	Pass																	
generates object code	first																	
reads the source code one line at a time	first																	
reads the source code one line at a time	second																	
removes white space	first																	
removes white space	second																	
adds labels to the symbol table	first																	
adds labels to the symbol table	second																	

9618_w22_qp_11_sc_6.c

QUESTION

(c) Tick (✓) **one or more** boxes in each row to indicate whether the task is performed in the first pass or the second pass of a two-pass assembler.

Task	First pass	Second pass
Remove comments.		
Read the assembly language program one line at a time.		
Generate the object code.		
Check the opcode is in the instruction set.		

[2]

MARK SCHEME

6(c)	1 mark for each pair of highlighted rows <table><tr><th data-bbox="300 297 906 342">Task</th><th data-bbox="906 297 1085 342">First pass</th><th data-bbox="1085 297 1308 342">Second pass</th></tr><tr><td data-bbox="300 342 906 416">Remove comments.</td><td data-bbox="906 342 1085 416">✓</td><td data-bbox="1085 342 1308 416"></td></tr><tr><td data-bbox="300 416 906 515">Read the assembly language program one line at a time.</td><td data-bbox="906 416 1085 515">✓</td><td data-bbox="1085 416 1308 515">✓</td></tr><tr><td data-bbox="300 515 906 584">Generate the object code.</td><td data-bbox="906 515 1085 584"></td><td data-bbox="1085 515 1308 584">✓</td></tr><tr><td data-bbox="300 584 906 649">Check the opcode is in the instruction set.</td><td data-bbox="906 584 1085 649">✓</td><td data-bbox="1085 584 1308 649"></td></tr></table>	Task	First pass	Second pass	Remove comments.	✓		Read the assembly language program one line at a time.	✓	✓	Generate the object code.		✓	Check the opcode is in the instruction set.	✓		2
Task	First pass	Second pass															
Remove comments.	✓																
Read the assembly language program one line at a time.	✓	✓															
Generate the object code.		✓															
Check the opcode is in the instruction set.	✓																

Trace a given simple assembly language program

9618_w25_qp_13_sc_6.a

QUESTION

- 6 The following table shows part of the instruction set for a processor. The processor has two registers: the Accumulator (ACC) and the Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to the ACC
AND	#n / Bn / &n	Bitwise AND operation of the contents of the ACC with the operand
AND	<address>	Bitwise AND operation of the contents of the ACC with the contents of <address>
XOR	#n / Bn / &n	Bitwise XOR operation of the contents of the ACC with the operand
XOR	<address>	Bitwise XOR operation of the contents of the ACC with the contents of <address>
OR	#n / Bn / &n	Bitwise OR operation of the contents of the ACC with the operand
OR	<address>	Bitwise OR operation of the contents of the ACC with the contents of <address>
CMP	#n	Compare the contents of the ACC with number n
CMP	<address>	Compare the contents of the ACC with the contents of <address>
LSL	#n	Bits in the ACC are shifted logically n places to the left. Zeros are introduced on the right-hand end
LSR	#n	Bits in the ACC are shifted logically n places to the right. Zeros are introduced on the left-hand end

<address> can be an absolute or symbolic address
denotes a denary number, e.g. #127
B denotes a binary number, e.g. B10010001
& denotes a hexadecimal number, e.g. &4A

The current contents of main memory are shown:

Address	Data
100	0000 0011
101	1010 1110
102	1100 1100
103	1111 1111
104	1100 1100

(a) Complete the table by writing the contents of the ACC after the execution of each instruction.

Current contents of the ACC	Instruction	Contents of the ACC after the execution of the instruction
0000 1111	AND 101	
0000 0000	LDM #100	
0000 0001	XOR &F1	
0001 0001	CMP 101	

[4]

MARK SCHEME

6(a)	1 mark for each correct row	4															
<table> <tr> <th>Current contents of the ACC</th><th>Instruction</th><th>Contents of the ACC after the execution of the instruction</th></tr> <tr> <td>0000 1111</td><td>AND 101</td><td>0000 1110</td></tr> <tr> <td>0000 0000</td><td>LDM #100</td><td>0110 0100</td></tr> <tr> <td>0000 0001</td><td>XOR &F1</td><td>1111 0000</td></tr> <tr> <td>0001 0001</td><td>CMP 101</td><td>0001 0001</td></tr> </table>			Current contents of the ACC	Instruction	Contents of the ACC after the execution of the instruction	0000 1111	AND 101	0000 1110	0000 0000	LDM #100	0110 0100	0000 0001	XOR &F1	1111 0000	0001 0001	CMP 101	0001 0001
Current contents of the ACC	Instruction	Contents of the ACC after the execution of the instruction															
0000 1111	AND 101	0000 1110															
0000 0000	LDM #100	0110 0100															
0000 0001	XOR &F1	1111 0000															
0001 0001	CMP 101	0001 0001															

QUESTION

- (b) The following table shows part of the instruction set for a processor. The processor has two registers: the ACC and an IX.

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDI	<address>	Indirect addressing. The address to be used is at the given address. Load the contents of this second address to ACC
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC
LDR	#n	Immediate addressing. Load the number n to IX
<address> can be an absolute or symbolic address # denotes a denary number, e.g. #127		

The current contents of the main memory and the index register are shown.

Address	Instruction
98	8
99	16
100	3
101	98
102	32
IX	2

Write the contents of the ACC after each instruction is executed.

Instruction	Value in ACC
LDM #98	
LDI 101	
LDX 100	

[3]

MARK SCHEME

3(b)	1 mark for each correct value, max 3 marks	3								
<table><tr><th>Instruction</th><th>Value in ACC</th></tr><tr><td>LDM #98</td><td>98</td></tr><tr><td>LDI 101</td><td>8</td></tr><tr><td>LDX 100</td><td>32</td></tr></table>			Instruction	Value in ACC	LDM #98	98	LDI 101	8	LDX 100	32
Instruction	Value in ACC									
LDM #98	98									
LDI 101	8									
LDX 100	32									

QUESTION

- 5 The following table shows part of the instruction set for a processor. The processor has two registers: the Accumulator (ACC) and an Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
STO	<address>	Store the contents of ACC at the given address
ADD	#n/Bn/&n	Add the number n to the ACC
ADD	<address>	Add the contents of the given address to the ACC
SUB	<address>	Subtract the contents of the given address from the ACC
SUB	#n/Bn/&n	Subtract the number n from the ACC
DEC	<register>	Subtract 1 from the contents of the register (ACC or IX)
JMP	<address>	Jump to the given address
CMP	<address>	Compare the contents of ACC with the contents of <address>
CMP	#n	Compare the contents of ACC with number n
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True
END		Return control to the operating system

ACC denotes Accumulator
 <address> can be an absolute or a symbolic address
 # denotes a denary number, e.g. #123
 B denotes a binary number, e.g. B01001010
 & denotes a hexadecimal number, e.g. &4A

- (a) The current contents of memory are:

Address	Instruction
50	47
51	48
52	49
53	50
...	
500	LDM #50
501	DEC ACC
502	CMP 51
503	JPE 505
504	JMP 501
505	STO 50
506	SUB #10
507	STO 51
508	END

(i) Trace the program currently in memory using the following trace table.

[illegible]

[3]

MARK SCHEME

5(a)(i)	1 mark for each shaded section					3
Instruction address	ACC	Memory address				
		50	51	52	53	
		47	48	49	50	
500	50					
501	49					
502						
504						
501	48					
502						
503						
505		48				
506	38					
507			38			

QUESTION

- 7 (a) The following table shows part of the instruction set for a processor. The processor has one register: the Accumulator (ACC).

Instruction		Explanation
Opcode	Operand	
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDM	#n	Immediate addressing. Load the number n to ACC
STO	<address>	Store the contents of ACC at the given address
ADD	#n/Bn/&n	Add the number n to the ACC
ADD	<address>	Add the contents of the given address to the ACC
INC	<register>	Add 1 to the contents of the register (ACC)
CMP	<address>	Compare the contents of ACC with the contents of <address>
CMP	#n	Compare the contents of ACC with number n
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True
JMP	<address>	Jump to the given address
END		Return control to the operating system

ACC denotes Accumulator
 <address> can be an absolute or a symbolic address
 # denotes a denary number, e.g. #123
 B denotes a binary number, e.g. B01001010
 & denotes a hexadecimal number, e.g. &4A

The current contents of memory are:

Address	Instruction
10	12
11	11
12	10
13	22
14	22
...	
100	LDD 10
101	ADD 12
102	STO 11
103	CMP 14
104	JPE 107
105	INC ACC
106	JMP 101
107	STO 10
108	END

- (i) Trace the program currently in memory using the following trace table.

Instruction address	ACC	Memory address				
		10	11	12	13	14
		12	11	10	22	22

[3]

MARK SCHEME

7(a)(i)

1 mark for each shaded section

3

Instruction address	ACC	Memory address				
		10	11	12	13	14
		12	11	10	22	22
100	12					
101	22					
102			22			
103						
104						
107		22				

QUESTION

- 8 (a) The following table shows part of the instruction set for a processor. The processor has two registers: the Accumulator (ACC) and an Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDI	<address>	Indirect addressing. The address to be used is at the given address. Load the contents of this second address to ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
STO	<address>	Store the contents of ACC at the given address
ADD	#n/Bn/&n	Add the number n to the ACC
DEC	<register>	Subtract 1 from the contents of the register (ACC or IX)
JMP	<address>	Jump to the given address
CMP	<address>	Compare the contents of ACC with the contents of <address>
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True
END		Return control to the operating system

ACC denotes Accumulator
 <address> can be an absolute or a symbolic address
 # denotes a denary number, e.g. #123
 B denotes a binary number, e.g. B01001010
 & denotes a hexadecimal number, e.g. &4A

The current contents of memory are:

address	Instruction
80	10
81	8
82	80
83	81
...	
200	LDD 81
201	INC ACC
202	STO 83
203	LDI 82
204	CMP 83
205	JPE 209
206	LDD 83
207	ADD #10
208	JMP 210
209	DEC ACC
210	STO 81
211	END

Trace the program currently in memory using the following trace table.

Instruction address	ACC	Memory address			
		80	81	82	83
		10	8	80	81

[4]

MARK SCHEME

8(a)

1 mark for each shaded section

4

Instruction address	ACC	Memory address			
		80	81	82	83
		10	8	80	81
200	8				
201	9				
202					9
203	10				
204					
206	9				
207	19				
208					
210			19		

QUESTION

- 7 The following table shows part of the instruction set for a processor. The processor has two registers, the Accumulator (ACC) and the Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDI	<address>	Indirect addressing. The address to be used is at the given address. Load the contents of this second address to ACC
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC
LDR	#n	Immediate addressing. Load the number n to IX
ADD	#n/Bn/&n	Add the number n to the ACC
ADD	<address>	Add the contents of the given address to the ACC
SUB	#n/Bn/&n	Subtract the number n from the ACC
SUB	<address>	Subtract the contents of the given address from the ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
DEC	<register>	Subtract 1 from the contents of the register (ACC or IX)
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001010 & denotes a hexadecimal number, e.g. &4A		

(a) The current contents of memory are shown:

Address	Data
50	54
51	55
52	50
53	52
54	100
55	25
56	50

The current contents of the ACC and IX are shown:

ACC	50
IX	45

Complete the table by writing the content of the ACC after each program has run.

	Instructions	ACC content
1	LDD 50 ADD #4 ADD 54	
2	LDI 53 DEC ACC ADD 56	
3	LDM #55 SUB #5	

[3]

MARK SCHEME

7(a)	1 mark each:	3												
	<table> <tr> <th></th><th>Instructions</th><th>ACC content</th></tr> <tr> <td>1</td><td>LDD 50 ADD #4 ADD 54</td><td>158</td></tr> <tr> <td>2</td><td>LDI 53 DEC ACC ADD 56</td><td>99</td></tr> <tr> <td>3</td><td>LDM #55 SUB #5</td><td>50</td></tr> </table>		Instructions	ACC content	1	LDD 50 ADD #4 ADD 54	158	2	LDI 53 DEC ACC ADD 56	99	3	LDM #55 SUB #5	50	
	Instructions	ACC content												
1	LDD 50 ADD #4 ADD 54	158												
2	LDI 53 DEC ACC ADD 56	99												
3	LDM #55 SUB #5	50												

QUESTION

- 8 The following table shows part of the instruction set for a processor. The processor has two registers, the Accumulator (ACC) and the Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDI	<address>	Indirect addressing. The address to be used is at the given address. Load the contents of this second address to ACC
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC
LDR	#n	Immediate addressing. Load the number n to IX
ADD	#n/Bn/&n	Add the number n to the ACC
ADD	<address>	Add the contents of the given address to the ACC
SUB	#n/Bn/&n	Subtract the number n from the ACC
SUB	<address>	Subtract the contents of the given address from the ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
DEC	<register>	Subtract 1 from the contents of the register (ACC or IX)
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001010 & denotes a hexadecimal number, e.g. &4A		

(a) The current contents of memory are shown:

Address	Data
19	25
20	23
21	2
22	4
23	15
24	50
25	22

The current contents of the ACC and IX are shown:

ACC	50
IX	20

Complete the table by writing the content of the ACC and the IX after each set of instructions has run.

	Instructions	ACC content	IX content
1	LDM #19 DEC ACC		
2	LDD 23 ADD 19		
3	LDI 25 INC ACC		
4	LDR #21 LDX 2		

[5]

MARK SCHEME

8(a)

1 mark for each correct content of the ACC (4)
1 mark for correct IX column

	Instructions	ACC content	IX content
1	LDM #19 DEC ACC	18	20
2	LDD 23 ADD 19	40	20
3	LDI 25 INC ACC	5	20
4	LDR #21 LDX 2	15	21

5

QUESTION

4

- 3 The following table shows part of the instruction set for a processor. The processor has two registers: the Accumulator (ACC) and an Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDI	<address>	Indirect addressing. The address to be used is at the given address. Load the contents of this second address to ACC
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC
LDR	#n	Immediate addressing. Load the number n to IX
ADD	#n/Bn/&n	Add the number n to the ACC
ADD	<address>	Add the contents of the given address to the ACC
DEC	<register>	Subtract 1 from the contents of the register (ACC or IX)
SUB	#n/Bn/&n	Subtract the number n from the ACC
SUB	<address>	Subtract the contents of the given address from the ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001010 & denotes a hexadecimal number, e.g. &4A		

(a) The current contents of memory are shown:

Address	Data
48	51
49	6
50	48
51	50
52	49
53	50
54	6

The current contents of the ACC and IX are shown:

ACC	2
IX	50

Complete the table by writing the content of the ACC after each program has run.

Program number	Code	ACC content
1	LDM #50 INC ACC SUB #1	
2	LDI 51 ADD 52	
3	LDR #2 LDX 50 DEC ACC	
4	LDD 52 SUB 54 INC ACC	

[4]

MARK SCHEME

3(a)	1 mark for each correct answer: <table border="1"> <thead> <tr> <th>Program Number</th><th>Code</th><th>ACC Content</th></tr> </thead> <tbody> <tr> <td>1</td><td>LDM #50 INC ACC SUB #1</td><td>50</td></tr> <tr> <td>2</td><td>LDI 51 ADD 52</td><td>97</td></tr> <tr> <td>3</td><td>LDR #2 LDX 50 DEC ACC</td><td>48</td></tr> <tr> <td>4</td><td>LDD 52 SUB 54 INC ACC</td><td>44</td></tr> </tbody> </table>	Program Number	Code	ACC Content	1	LDM #50 INC ACC SUB #1	50	2	LDI 51 ADD 52	97	3	LDR #2 LDX 50 DEC ACC	48	4	LDD 52 SUB 54 INC ACC	44	4
Program Number	Code	ACC Content															
1	LDM #50 INC ACC SUB #1	50															
2	LDI 51 ADD 52	97															
3	LDR #2 LDX 50 DEC ACC	48															
4	LDD 52 SUB 54 INC ACC	44															

9618_s24_qp_12_sc_5.a

QUESTION

- 5 The following table shows part of the instruction set for a processor. The processor has two registers: the Accumulator (ACC) and an Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDI	<address>	Indirect addressing. The address to be used is at the given address. Load the contents of this second address to ACC
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC
LDR	#n	Immediate addressing. Load the number n to IX
ADD	#n/Bn/&n	Add the number n to the ACC
ADD	<address>	Add the contents of the given address to the ACC
SUB	#n/Bn/&n	Subtract the number n from the ACC
SUB	<address>	Subtract the contents of the given address from the ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001010 & denotes a hexadecimal number, e.g. &4A		

(a) The current contents of memory are shown:

Address	Data
10	1
11	3
12	5
13	11
14	10
15	16
16	12

The current contents of the ACC and IX are shown:

ACC	10
IX	0

Complete the table by writing the content of the ACC after each program has run.

Program number	Code	ACC content
1	LDI 15 SUB #1	
2	LDD 14 ADD 11	
3	LDM #11 ADD #3 SUB 16	
4	LDR #2 LDX 14 ADD #2	

[4]

MARK SCHEME

5(a)	1 mark for each correct answer:	4															
	<table> <tr> <th>Program Number</th><th>Code</th><th>ACC Content</th></tr> <tr> <td>1</td><td>LDI 15 SUB #1</td><td>11</td></tr> <tr> <td>2</td><td>LDD 14 ADD 11</td><td>13</td></tr> <tr> <td>3</td><td>LDM #11 ADD #3 SUB 16</td><td>2</td></tr> <tr> <td>4</td><td>LDR #2 LDX 14 ADD #2</td><td>14</td></tr> </table>	Program Number	Code	ACC Content	1	LDI 15 SUB #1	11	2	LDD 14 ADD 11	13	3	LDM #11 ADD #3 SUB 16	2	4	LDR #2 LDX 14 ADD #2	14	
Program Number	Code	ACC Content															
1	LDI 15 SUB #1	11															
2	LDD 14 ADD 11	13															
3	LDM #11 ADD #3 SUB 16	2															
4	LDR #2 LDX 14 ADD #2	14															

9618_s24_qp_11_sc_4.a

QUESTION

- 4 The following table shows part of the instruction set for a processor. The processor has two registers: the Accumulator (ACC) and an Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDI	<address>	Indirect addressing. The address to be used is at the given address. Load the contents of this second address to ACC
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the Index Register. Copy the contents of this calculated address to ACC
LDR	#n	Immediate addressing. Load the number n to IX
ADD	#n/Bn/&n	Add the number n to the ACC
ADD	<address>	Add the contents of the given address to the ACC
SUB	#n/Bn/&n	Subtract the number n from the ACC
SUB	<address>	Subtract the contents of the given address from the ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001010 & denotes a hexadecimal number, e.g. &4A		

(a) The current contents of memory are shown:

Address	Data
19	24
20	2
21	1
22	3
23	5
24	4
25	22

The current contents of the ACC and IX are shown:

ACC	12
IX	1

Complete the table by writing the content of the ACC after each program has run.

Program number	Code	ACC content
1	LDD 20 ADD #2	
2	LDX 22	
3	LDI 25 INC ACC SUB 22	
4	LDD 19 LDM #5 LDM #25	

[4]

MARK SCHEME

4(a)	1 mark for each correct answer: <table> <tr> <th>Program Number</th><th>Code</th><th>ACC Content</th></tr> <tr> <td>1</td><td>LDD 20 ADD #2</td><td>4</td></tr> <tr> <td>2</td><td>LDX 22</td><td>5</td></tr> <tr> <td>3</td><td>LDI 25 INC ACC SUB 22</td><td>1</td></tr> <tr> <td>4</td><td>LDD 19 LDM #5 LDM #25</td><td>25</td></tr> </table>	Program Number	Code	ACC Content	1	LDD 20 ADD #2	4	2	LDX 22	5	3	LDI 25 INC ACC SUB 22	1	4	LDD 19 LDM #5 LDM #25	25	4
Program Number	Code	ACC Content															
1	LDD 20 ADD #2	4															
2	LDX 22	5															
3	LDI 25 INC ACC SUB 22	1															
4	LDD 19 LDM #5 LDM #25	25															

QUESTION

- 6 The following table shows part of the instruction set for a processor. The processor has one general purpose register, the Accumulator (ACC), and an Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC
LDR	#n	Immediate addressing. Load the number n to IX
MOV	<register>	Moves the contents of the accumulator to the given register (IX)
STO	<address>	Store contents of ACC at the given address
ADD	<address>	Add the contents of the given address to the ACC
ADD	#n	Add the denary number n to the ACC
SUB	#n	Subtract the denary number n from the ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
JMP	<address>	Jump to the given address
CMP	#n	Compare the contents of ACC with number n
CMI	<address>	Indirect addressing. The address to be used is at the given address. Compare the contents of ACC with the contents of this second address
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True
JPN	<address>	Following a compare instruction, jump to <address> if the compare was False
OUT		Output to the screen the character whose ASCII value is stored in ACC
END		Return control to the operating system
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001101		

(a) The current contents of main memory and selected values from the ASCII character set are given.

(i) Trace the program currently in memory using the trace table.

Address	Instruction	Instruction address	ACC	IX	Memory address					Output
					100	101	110	111	112	
75	LDR #0				1	0	97	98	97	
76	LDD 100									
77	CMP #2									
78	JPE 91									
79	LDX 110									
80	SUB #32									
81	CMP #65									
82	JPN 86									
83	LDM #1									
84	ADD 101									
85	STO 101									
86	LDM #1									
87	ADD 100									
88	STO 100									
89	INC IX									
90	JMP 76									
91	LDD 101									
92	ADD #48									
93	OUT									
94	END									
...										
100	1									
101	0									
...										
110	97									
111	98									
112	97									

ASCII value	Character
49	1
50	2
51	3
52	4
	
65	A
66	B
67	C
68	D

[4]

MARK SCHEME

6(a)(i)

1 mark for each set of highlighted rows:

4

Instruction address	ACC	IX	Memory address					Output
			100	101	110	111	112	
			1	0	97	98	97	
75		0						
76	1							
77								
78								
79	97							
80	65							
81								
82								
83	1							
84								
85				1				
86								
87	2							
88			2					
89		1						
90								
76								
77								
78								
91	1							
92	49							
93								1
94								

QUESTION

- 7 The following table shows part of the instruction set for a processor. The processor has one general purpose register, the Accumulator (ACC), and an Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC
LDR	#n	Immediate addressing. Load the number n to IX
STO	<address>	Store the contents of ACC at the given address
ADD	<address>	Add the contents of the given address to the ACC
ADD	#n	Add the denary number n to the ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
JMP	<address>	Jump to the given address
CMP	<address>	Compare the contents of ACC with the contents of <address>
CMI	<address>	Indirect addressing. The address to be used is at the given address. Compare the contents of ACC with the contents of this second address
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True
JPN	<address>	Following a compare instruction, jump to <address> if the compare was False
END		Return control to the operating system
<address> can be an absolute or symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001101		

- (a) Trace the program currently in memory using the trace table, stopping when line 90 is executed for a second time.

Address	Instruction	Instruction address	ACC	IX	Memory address						
					100	101	102	103	110	111	112
75	LDR #0				0	0	112	4	1	4	0
76	LDX 110										
77	CMI 102										
78	JPE 91										
79	CMP 103										
80	JPN 84										
81	ADD 101										
82	STO 101										
83	JMP 86										
84	INC ACC										
85	STO 101										
86	LDD 100										
87	INC ACC										
88	STO 100										
89	INC IX										
90	JMP 76										
91	END										
...	↗										
100	0										
101	0										
102	112										
103	4										
...	↗										
110	1										
111	4										
112	0										

[5]

MARK SCHEME

7(a)

1 mark for each set of highlighted rows.

5

Instruction address	ACC	IX	Memory address						
			100	101	102	103	110	111	112
			0	0	112	4	1	4	0
75		0							
76	1								
77									
78									
79									
80									
84	2								
85				2					
86	0								
87	1								
88			1						
89		1							
90									
76	4								
77									
78									
79									
80									
81	6								
82				6					
83									
86	1								
87	2								
88			2						
89		2							
90									

QUESTION

- 6 The following table shows part of the instruction set for a processor. The processor has one general purpose register, the Accumulator (ACC), and an Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC
LDR	#n	Immediate addressing. Load the number n to IX
STO	<address>	Store contents of ACC at the given address
ADD	<address>	Add the contents of the given address to the ACC
ADD	#n	Add the denary number n to the ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
CMP	#n	Compare the contents of ACC with number n
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True
JPN	<address>	Following a compare instruction, jump to <address> if the compare was False
OUT		Output to the screen the character whose ASCII value is stored in ACC
END		Return control to the operating system
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001101		

- (i) Trace the program currently in memory using the trace table.

[4]

MARK SCHEME

6(a)(i)

1 mark for each set of highlighted rows.

4

Instruction n address	ACC	IX	Memory address					Output
			100	101	110	111	112	
			0	0	66	65	35	
77		0						
78	66							
79								
80								
81								
82				66				
83	1							
84								
85			1					
86		1						
87	65							
88								
89								
81	66							
82								
83	1							
84	2							
85			2					
86		2						
87	35							
88								
89								
90	2							
91	50							
92								2
93								

QUESTION

(ii) The following instructions are repeated for your reference.

Instruction		Explanation
Opcode	Operand	
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
STO	<address>	Store contents of ACC at the given address

State the purpose of this part of an assembly language program.

```
LDD 100
STO 165
LDD 101
STO 100
LDD 165
STO 101
```

.....

..... [1]

MARK SCHEME

6(a)(ii)	swaps the contents of memory address <u>100</u> and <u>101</u>	1
----------	--	----------

QUESTION

- 4 The table shows part of the instruction set for a processor. The processor has one general purpose register, the Accumulator (ACC), and an Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
STO	<address>	Store contents of ACC at the given address
ADD	<address>	Add the contents of the given address to the ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
DEC	<register>	Subtract 1 from the contents of the register (ACC or IX)
CMP	<address>	Compare the contents of ACC with the contents of <address>
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True
JPN	<address>	Following a compare instruction, jump to <address> if the compare was False
JMP	<address>	Jump to the given address
IN		Key in a character and store its ASCII value in ACC
OUT		Output to the screen the character whose ASCII value is stored in ACC
END		Return control to the operating system

denotes a denary number, e.g. #123

The current contents of the main memory and selected values from the ASCII character set are:

Address	Instruction
70	IN
71	CMP 100
72	JPE 80
73	CMP 101
74	JPE 76
75	JMP 80
76	LDD 102
77	INC ACC
78	STO 102
79	JMP 70
80	LDD 102
81	DEC ACC
82	STO 102
83	JMP 70
...	
100	68
101	65
102	100

ASCII code table (selected codes only)

ASCII code	Character
65	A
66	B
67	C
68	D

- A D

[illegible]

Page 38 of 73

MARK SCHEME

4(a)

1 mark for each shaded section / bullet point

4

- Load 65 into ACC
- Load 100 into ACC, increment and store in 102
- Load 68 into ACC
- Load 101 into ACC, decrement and store in 102

Instruction address	ACC	Memory address		
		100	101	102
		68	65	100
70	65			
71				
72				
73				
74				
76	100			
77	101			
78				101
79				
70	68			
71				
72				
80	101			
81	100			
82				100
83				
(70)				

QUESTION

- (b) The following table shows part of the instruction set for a processor. The processor has one general purpose register, the Accumulator (ACC), and an Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDI	<address>	Indirect addressing: The address to be used is at the given address. Load the contents of this second address to ACC
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the Index Register. Copy the contents of this calculated address to ACC
LDR	#n	Immediate addressing. Load the number n to IX
MOV	<register>	Move the contents of the accumulator to the given register (IX)
STO	<address>	Store contents of ACC at the given address
ADD	<address>	Add the contents of the given address to the ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
CMP	<address>	Compare the contents of ACC with the contents of <address>
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True
JPN	<address>	Following a compare instruction, jump to <address> if the compare was False
JMP	<address>	Jump to the given address
OUT		Output to the screen the character whose ASCII value is stored in ACC
END		Return control to the operating system
LSL	#n	Bits in ACC are shifted logically n places to the left. Zeros are introduced on the right hand end
LSR	#n	Bits in ACC are shifted logically n places to the right. Zeros are introduced on the left hand end
<address> can be an absolute address or a symbolic address # denotes a denary number, e.g. #123		

The current contents of the main memory and selected values from the ASCII character set are shown.

Address	Instruction
200	LDD 365
201	CMP 366
202	JPE 209
203	INC ACC
204	STO 365
205	MOV IX
206	LDX 365
207	OUT
208	JMP 200
209	END
...	
365	1
366	3
367	65
368	66
IX	0

ASCII code table (selected codes only)	
ASCII code	Character
65	A
66	B
67	C
68	D

Complete the trace table for the program currently in main memory.

[illegible]

[6]

MARK SCHEME

3(b)

1 mark for each shaded set of values

6

Instruction address	ACC	Memory address				IX	Output
		365	366	367	368		
		1	3	65	66	0	
200	1						
201							
202							
203	2						
204		2					
205						2	
206	65						
207							A
208							
200	2						
201							
202							
203	3						
204		3					
205						3	
206	66						
207							B
208							
200	3						
201							
202							
209							

Show understanding that a set of instructions are grouped

9618_w25_qp_13_sc_6.c

QUESTION

(c) Assembly language instructions are grouped.

Complete each statement by writing the name of the appropriate instruction group.

Loading data into the accumulator is an example of an instruction in the

..... group. Incrementing the index register is an example

of an instruction in the group. Branching to another

address is an example of an instruction in the group.
[3]

MARK SCHEME

6(c)	1 mark for each bullet point, max 3 marks • Data movement • Arithmetic operations • Conditional and unconditional (jump) instructions	3
------	---	----------

QUESTION

- 9 The following table shows part of the instruction set for a processor. The processor has two registers, the Accumulator (ACC) and the Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC
LDR	#n	Immediate addressing. Load the number n to IX
STO	<address>	Store the contents of ACC at the given address
ADD	#n	Add the denary number n to the ACC
JMP	<address>	Jump to the given address
INC	<register>	Add 1 to the contents of the register (ACC or IX)
CMP	<address>	Compare the contents of ACC with the contents of <address>
CMI	<address>	Indirect addressing. The address to be used is at the given address. Compare the contents of ACC with the contents of this second address
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True
IN		Key in a character and store its ASCII value in ACC
OUT		Output to the screen the character whose ASCII value is stored in ACC
END		Return control to the operating system
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123		

- (a) The instructions in the processor's instruction set can be grouped according to their function. Identify the instruction group for each of the following opcodes.

IN

ADD

JPE

CMI

[4]

MARK SCHEME

9(a)	1 mark for each bullet point - IN - Input and output of data - ADD - Arithmetic operations - JPE - Unconditional and conditional instructions - CMI - Compare instructions	4
------	---	----------

9618_w23_qp_11_sc_8.b.i

QUESTION

- (b) The following table shows part of the instruction set for a processor. The processor has two registers, the Accumulator (ACC) and the Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDR	#n	Immediate addressing. Load the number n to IX
STO	<address>	Store contents of ACC at the given address
ADD	<address>	Add the contents of the given address to the ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
CMP	#n	Compare the contents of ACC with number n
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True
OUT		Output to the screen the character whose ASCII value is stored in ACC

<address> can be an absolute or symbolic address
denotes a denary number, e.g. #123

- (i) Give **one** example of an instruction that belongs to **each** of the following instruction groups.

Only use the instructions given in the table. Each instruction must have a suitable operand.

Data movement

Arithmetic operation

Conditional instruction

[3]

MARK SCHEME

8(b)(i)	1 mark for each bullet point - Data movement: e.g. LDR #50 // STO 201 - Arithmetic operation: e.g. ADD 100 // INC IX - Conditional instruction: e.g. JPE 96	3
---------	--	----------

QUESTION

(c) One instruction group is data movement.

Give the name of **one other** instruction group.

.....
 [1]

MARK SCHEME

6(c)	1 mark for a correct name: - input and output of data - arithmetic operations - unconditional and conditional instructions - compare instructions	1
------	---	----------

QUESTION

(ii) The following instructions are repeated for your reference.

Instruction		Explanation
Opcode	Operand	
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
STO	<address>	Store contents of ACC at the given address

State the purpose of this part of an assembly language program.

```
LDD 100
STO 165
LDD 101
STO 100
LDD 165
STO 101
```

.....
 [1]

MARK SCHEME

6(a)(ii)	swaps the contents of memory address <u>100</u> and <u>101</u>	1
----------	--	----------

QUESTION

(ii) The following table contains five assembly language instruction groups.

Write an appropriate assembly language instruction for each instruction group, using the given instruction set. The first one has been completed for you.

Instruction Group	Instruction
Data movement	LDM #2
Input and output of data	
Arithmetic operations	
Unconditional and conditional instructions	
Compare instructions	

[4]

MARK SCHEME

8(b)(ii)	1 mark for each correct instruction	4												
<table><tr><th>Instruction Group</th><th>Instruction</th></tr><tr><td>Data movement</td><td>LDM #2</td></tr><tr><td>Input and output of data</td><td>IN / OUT</td></tr><tr><td>Arithmetic Operations</td><td>INC ACC / INC IX</td></tr><tr><td>Unconditional and conditional instructions</td><td>JPN 100 / JMP 100</td></tr><tr><td>Compare instructions</td><td>CMP 100</td></tr></table>			Instruction Group	Instruction	Data movement	LDM #2	Input and output of data	IN / OUT	Arithmetic Operations	INC ACC / INC IX	Unconditional and conditional instructions	JPN 100 / JMP 100	Compare instructions	CMP 100
Instruction Group	Instruction													
Data movement	LDM #2													
Input and output of data	IN / OUT													
Arithmetic Operations	INC ACC / INC IX													
Unconditional and conditional instructions	JPN 100 / JMP 100													
Compare instructions	CMP 100													

Show understanding of and be able to use different modes of addressing

9618_w25_qp_12_sc_3.a.i

QUESTION

3 (a) (i) State what is meant by relative addressing.

.....
.....
..... [1]

MARK SCHEME

3(a)(i)	1 mark The value of the operand is an offset value which is added to another base value to give the address from which the contents are loaded to the accumulator	1
---------	--	----------

QUESTION

- (b) The following table shows part of the instruction set for a processor. The processor has two registers: the ACC and an IX.

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDI	<address>	Indirect addressing. The address to be used is at the given address. Load the contents of this second address to ACC
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC
LDR	#n	Immediate addressing. Load the number n to IX
<address> can be an absolute or symbolic address # denotes a denary number, e.g. #127		

The current contents of the main memory and the index register are shown.

Address	Instruction
98	8
99	16
100	3
101	98
102	32
IX	2

Write the contents of the ACC after each instruction is executed.

Instruction	Value in ACC
LDM #98	
LDI 101	
LDX 100	

[3]

MARK SCHEME

3(b)	1 mark for each correct value, max 3 marks	3								
<table><tr><th>Instruction</th><th>Value in ACC</th></tr><tr><td>LDM #98</td><td>98</td></tr><tr><td>LDI 101</td><td>8</td></tr><tr><td>LDX 100</td><td>32</td></tr></table>			Instruction	Value in ACC	LDM #98	98	LDI 101	8	LDX 100	32
Instruction	Value in ACC									
LDM #98	98									
LDI 101	8									
LDX 100	32									

QUESTION

- (ii) Complete the table by identifying **and** describing **two** modes of addressing that are **not** used in the program in part (a).

Mode of addressing	Description
.....	
.....	

[4]

MARK SCHEME

5(a)(ii)	1 mark for addressing mode, 1 mark for matching description • Direct • The operand is the address of the data • Indirect • The operand points to the memory location which is the address of the data • Indexed • The address of the data is formed by adding the contents of the Index Register (IX) to the operand	4
----------	---	----------

QUESTION

- (ii) State the effect of changing instruction `LDD 10` in address 100 to `LDM #10`

.....

 [1]

MARK SCHEME

7(a)(ii)	1 mark from: - The number 10 will be loaded into the ACC instead of 12 / contents of address 10 - The addition will give 20 not 22 so the comparison will fail and result in an infinite loop	1
----------	---	----------

9618_s25_qp_12_sc_7.a.iii

QUESTION

(iii) Identify **and** describe **one** mode of addressing **not** given in the table of instructions in part (a).

Mode of addressing

Description

.....

.....

[2]

MARK SCHEME

7(a)(iii)	1 mark for name of mode, 1 mark for matching description e.g. - Indirect - The operand points to the memory location that contains the address of the data - Indexed - The address of the data is formed by adding the contents of the Index Register (IX) to the operand - Relative - The address is calculated using its distance from a base address	2
-----------	---	----------

QUESTION

- 8 (a) The following table shows part of the instruction set for a processor. The processor has two registers: the Accumulator (ACC) and an Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDI	<address>	Indirect addressing. The address to be used is at the given address. Load the contents of this second address to ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
STO	<address>	Store the contents of ACC at the given address
ADD	#n/Bn/&n	Add the number n to the ACC
DEC	<register>	Subtract 1 from the contents of the register (ACC or IX)
JMP	<address>	Jump to the given address
CMP	<address>	Compare the contents of ACC with the contents of <address>
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True
END		Return control to the operating system

ACC denotes Accumulator
 <address> can be an absolute or a symbolic address
 # denotes a denary number, e.g. #123
 B denotes a binary number, e.g. B01001010
 & denotes a hexadecimal number, e.g. &4A

The current contents of memory are:

address	Instruction
80	10
81	8
82	80
83	81
...	
200	LDD 81
201	INC ACC
202	STO 83
203	LDI 82
204	CMP 83
205	JPE 209
206	LDD 83
207	ADD #10
208	JMP 210
209	DEC ACC
210	STO 81
211	END

Trace the program currently in memory using the following trace table.

Instruction address	ACC	Memory address			
		80	81	82	83
		10	8	80	81

[4]

MARK SCHEME

8(a)	1 mark for each shaded section					4
Instruction address	ACC	Memory address				
		80	81	82	83	
		10	8	80	81	
200	8					
201	9					
202					9	
203	10					
204						
206	9					
207	19					
208						
210			19			

QUESTION

- 7 The following table shows part of the instruction set for a processor. The processor has two registers, the Accumulator (ACC) and the Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDI	<address>	Indirect addressing. The address to be used is at the given address. Load the contents of this second address to ACC
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC
LDR	#n	Immediate addressing. Load the number n to IX
ADD	#n/Bn/&n	Add the number n to the ACC
ADD	<address>	Add the contents of the given address to the ACC
SUB	#n/Bn/&n	Subtract the number n from the ACC
SUB	<address>	Subtract the contents of the given address from the ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
DEC	<register>	Subtract 1 from the contents of the register (ACC or IX)
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001010 & denotes a hexadecimal number, e.g. &4A		

(a) The current contents of memory are shown:

Address	Data
50	54
51	55
52	50
53	52
54	100
55	25
56	50

The current contents of the ACC and IX are shown:

ACC	50
IX	45

Complete the table by writing the content of the ACC after each program has run.

	Instructions	ACC content
1	LDD 50 ADD #4 ADD 54	
2	LDI 53 DEC ACC ADD 56	
3	LDM #55 SUB #5	

[3]

MARK SCHEME

7(a)	1 mark each:	3												
	<table> <tr> <th></th><th>Instructions</th><th>ACC content</th></tr> <tr> <td>1</td><td>LDD 50 ADD #4 ADD 54</td><td>158</td></tr> <tr> <td>2</td><td>LDI 53 DEC ACC ADD 56</td><td>99</td></tr> <tr> <td>3</td><td>LDM #55 SUB #5</td><td>50</td></tr> </table>		Instructions	ACC content	1	LDD 50 ADD #4 ADD 54	158	2	LDI 53 DEC ACC ADD 56	99	3	LDM #55 SUB #5	50	
	Instructions	ACC content												
1	LDD 50 ADD #4 ADD 54	158												
2	LDI 53 DEC ACC ADD 56	99												
3	LDM #55 SUB #5	50												

QUESTION

- 8 The following table shows part of the instruction set for a processor. The processor has two registers, the Accumulator (ACC) and the Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDI	<address>	Indirect addressing. The address to be used is at the given address. Load the contents of this second address to ACC
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC
LDR	#n	Immediate addressing. Load the number n to IX
ADD	#n/Bn/&n	Add the number n to the ACC
ADD	<address>	Add the contents of the given address to the ACC
SUB	#n/Bn/&n	Subtract the number n from the ACC
SUB	<address>	Subtract the contents of the given address from the ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
DEC	<register>	Subtract 1 from the contents of the register (ACC or IX)
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001010 & denotes a hexadecimal number, e.g. &4A		

(a) The current contents of memory are shown:

Address	Data
19	25
20	23
21	2
22	4
23	15
24	50
25	22

The current contents of the ACC and IX are shown:

ACC	50
IX	20

Complete the table by writing the content of the ACC and the IX after each set of instructions has run.

	Instructions	ACC content	IX content
1	LDM #19 DEC ACC		
2	LDD 23 ADD 19		
3	LDI 25 INC ACC		
4	LDR #21 LDX 2		

[5]

MARK SCHEME

8(a)

1 mark for each correct content of the ACC (4)
1 mark for correct IX column

	Instructions	ACC content	IX content
1	LDM #19 DEC ACC	18	20
2	LDD 23 ADD 19	40	20
3	LDI 25 INC ACC	5	20
4	LDR #21 LDX 2	15	21

5

QUESTION

4

- 3 The following table shows part of the instruction set for a processor. The processor has two registers: the Accumulator (ACC) and an Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDI	<address>	Indirect addressing. The address to be used is at the given address. Load the contents of this second address to ACC
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC
LDR	#n	Immediate addressing. Load the number n to IX
ADD	#n/Bn/&n	Add the number n to the ACC
ADD	<address>	Add the contents of the given address to the ACC
DEC	<register>	Subtract 1 from the contents of the register (ACC or IX)
SUB	#n/Bn/&n	Subtract the number n from the ACC
SUB	<address>	Subtract the contents of the given address from the ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001010 & denotes a hexadecimal number, e.g. &4A		

(a) The current contents of memory are shown:

Address	Data
48	51
49	6
50	48
51	50
52	49
53	50
54	6

The current contents of the ACC and IX are shown:

ACC	2
IX	50

Complete the table by writing the content of the ACC after each program has run.

Program number	Code	ACC content
1	LDM #50 INC ACC SUB #1	
2	LDI 51 ADD 52	
3	LDR #2 LDX 50 DEC ACC	
4	LDD 52 SUB 54 INC ACC	

[4]

MARK SCHEME

3(a)	1 mark for each correct answer: <table> <tr> <th>Program Number</th><th>Code</th><th>ACC Content</th></tr> <tr> <td>1</td><td>LDM #50 INC ACC SUB #1</td><td>50</td></tr> <tr> <td>2</td><td>LDI 51 ADD 52</td><td>97</td></tr> <tr> <td>3</td><td>LDR #2 LDX 50 DEC ACC</td><td>48</td></tr> <tr> <td>4</td><td>LDD 52 SUB 54 INC ACC</td><td>44</td></tr> </table>	Program Number	Code	ACC Content	1	LDM #50 INC ACC SUB #1	50	2	LDI 51 ADD 52	97	3	LDR #2 LDX 50 DEC ACC	48	4	LDD 52 SUB 54 INC ACC	44	4
Program Number	Code	ACC Content															
1	LDM #50 INC ACC SUB #1	50															
2	LDI 51 ADD 52	97															
3	LDR #2 LDX 50 DEC ACC	48															
4	LDD 52 SUB 54 INC ACC	44															

9618_s24_qp_12_sc_5.a

QUESTION

- 5 The following table shows part of the instruction set for a processor. The processor has two registers: the Accumulator (ACC) and an Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDI	<address>	Indirect addressing. The address to be used is at the given address. Load the contents of this second address to ACC
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC
LDR	#n	Immediate addressing. Load the number n to IX
ADD	#n/Bn/&n	Add the number n to the ACC
ADD	<address>	Add the contents of the given address to the ACC
SUB	#n/Bn/&n	Subtract the number n from the ACC
SUB	<address>	Subtract the contents of the given address from the ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001010 & denotes a hexadecimal number, e.g. &4A		

(a) The current contents of memory are shown:

Address	Data
10	1
11	3
12	5
13	11
14	10
15	16
16	12

The current contents of the ACC and IX are shown:

ACC	10
IX	0

Complete the table by writing the content of the ACC after each program has run.

Program number	Code	ACC content
1	LDI 15 SUB #1	
2	LDD 14 ADD 11	
3	LDM #11 ADD #3 SUB 16	
4	LDR #2 LDX 14 ADD #2	

[4]

MARK SCHEME

5(a)	1 mark for each correct answer:	4															
	<table> <tr> <th>Program Number</th><th>Code</th><th>ACC Content</th></tr> <tr> <td>1</td><td>LDI 15 SUB #1</td><td>11</td></tr> <tr> <td>2</td><td>LDD 14 ADD 11</td><td>13</td></tr> <tr> <td>3</td><td>LDM #11 ADD #3 SUB 16</td><td>2</td></tr> <tr> <td>4</td><td>LDR #2 LDX 14 ADD #2</td><td>14</td></tr> </table>	Program Number	Code	ACC Content	1	LDI 15 SUB #1	11	2	LDD 14 ADD 11	13	3	LDM #11 ADD #3 SUB 16	2	4	LDR #2 LDX 14 ADD #2	14	
Program Number	Code	ACC Content															
1	LDI 15 SUB #1	11															
2	LDD 14 ADD 11	13															
3	LDM #11 ADD #3 SUB 16	2															
4	LDR #2 LDX 14 ADD #2	14															

9618_s24_qp_11_sc_4.a

QUESTION

- 4 The following table shows part of the instruction set for a processor. The processor has two registers: the Accumulator (ACC) and an Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDI	<address>	Indirect addressing. The address to be used is at the given address. Load the contents of this second address to ACC
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the Index Register. Copy the contents of this calculated address to ACC
LDR	#n	Immediate addressing. Load the number n to IX
ADD	#n/Bn/&n	Add the number n to the ACC
ADD	<address>	Add the contents of the given address to the ACC
SUB	#n/Bn/&n	Subtract the number n from the ACC
SUB	<address>	Subtract the contents of the given address from the ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001010 & denotes a hexadecimal number, e.g. &4A		

(a) The current contents of memory are shown:

Address	Data
19	24
20	2
21	1
22	3
23	5
24	4
25	22

The current contents of the ACC and IX are shown:

ACC	12
IX	1

Complete the table by writing the content of the ACC after each program has run.

Program number	Code	ACC content
1	LDD 20 ADD #2	
2	LDX 22	
3	LDI 25 INC ACC SUB 22	
4	LDD 19 LDM #5 LDM #25	

[4]

MARK SCHEME

4(a)	1 mark for each correct answer: <table> <tr> <th>Program Number</th><th>Code</th><th>ACC Content</th></tr> <tr> <td>1</td><td>LDD 20 ADD #2</td><td>4</td></tr> <tr> <td>2</td><td>LDX 22</td><td>5</td></tr> <tr> <td>3</td><td>LDI 25 INC ACC SUB 22</td><td>1</td></tr> <tr> <td>4</td><td>LDD 19 LDM #5 LDM #25</td><td>25</td></tr> </table>	Program Number	Code	ACC Content	1	LDD 20 ADD #2	4	2	LDX 22	5	3	LDI 25 INC ACC SUB 22	1	4	LDD 19 LDM #5 LDM #25	25	4
Program Number	Code	ACC Content															
1	LDD 20 ADD #2	4															
2	LDX 22	5															
3	LDI 25 INC ACC SUB 22	1															
4	LDD 19 LDM #5 LDM #25	25															

QUESTION

(ii) The instruction `LDR #2` uses immediate addressing.

Give **one** similarity and **one** difference between direct addressing and indexed addressing.

Similarity

.....

.....

Difference

.....

.....

[2]

MARK SCHEME

8(b)(ii)	1 mark for each bullet point (max 2) Similarity: - both load the contents of an <u>address</u> into the <u>Accumulator</u> Difference: - direct accesses the address given by the operand whereas indexed adds the contents of IX to the operand and accesses the data at that calculated address	2
----------	---	----------

QUESTION

(iii) Identify **one other** mode of addressing.

.....

..... [1]

MARK SCHEME

8(b)(iii)	1 mark for - Indirect (addressing) - Relative (addressing)	1
-----------	---	----------

QUESTION

(b) Assembly language statements can use different modes of addressing.

Complete the following description of addressing modes.

..... addressing is when the operand holds the memory address of the data.

..... addressing is when the operand holds a memory address that stores the memory address of the data.

..... addressing is when the operand is the data.

[3]

MARK SCHEME

3(b)	1 mark for each correct term Direct addressing is when the operand holds the memory address of the data. Indirect addressing is when the operand holds a memory address that stores the memory address of the data. Immediate addressing is when the operand is the data.	3
------	--	----------

QUESTION

(ii) Explain the purpose of **relative addressing** in an assembly language program.

.....

 [2]

MARK SCHEME

6(a)(ii)	1 mark for each bullet point: - To allow for re-locatable code ... because all (target) addresses can be specified by the base address + offset	2
----------	---	----------

QUESTION

(iii) The opcode `LDM` uses immediate addressing. The opcode `LDD` uses direct addressing.

Identify **and** describe **one additional** mode of addressing.

Mode of addressing

Description

.....

.....

[2]

MARK SCHEME

8(b)(iii)	1 mark for name, 1 mark for description • Indirect addressing • the address to be used is at the given address • Relative addressing • the address to be used is an offset number of locations away, relative to the address of the current instruction • Indexed addressing • form the address from the given address plus the contents of the index register	2
-----------	--	----------

QUESTION

- (b) The following table shows part of the instruction set for a processor. The processor has one general purpose register, the Accumulator (ACC), and an Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDI	<address>	Indirect addressing: The address to be used is at the given address. Load the contents of this second address to ACC
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the Index Register. Copy the contents of this calculated address to ACC
LDR	#n	Immediate addressing. Load the number n to IX
MOV	<register>	Move the contents of the accumulator to the given register (IX)
STO	<address>	Store contents of ACC at the given address
ADD	<address>	Add the contents of the given address to the ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
CMP	<address>	Compare the contents of ACC with the contents of <address>
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True
JPN	<address>	Following a compare instruction, jump to <address> if the compare was False
JMP	<address>	Jump to the given address
OUT		Output to the screen the character whose ASCII value is stored in ACC
END		Return control to the operating system
LSL	#n	Bits in ACC are shifted logically n places to the left. Zeros are introduced on the right hand end
LSR	#n	Bits in ACC are shifted logically n places to the right. Zeros are introduced on the left hand end
<address> can be an absolute address or a symbolic address # denotes a denary number, e.g. #123		

The current contents of the main memory and selected values from the ASCII character set are shown.

Address	Instruction
200	LDD 365
201	CMP 366
202	JPE 209
203	INC ACC
204	STO 365
205	MOV IX
206	LDX 365
207	OUT
208	JMP 200
209	END
...	
365	1
366	3
367	65
368	66
IX	0

ASCII code table (selected codes only)	
ASCII code	Character
65	A
66	B
67	C
68	D

Complete the trace table for the program currently in main memory.

[illegible]

[6]

MARK SCHEME

3(b)

1 mark for each shaded set of values

6

Instruction address	ACC	Memory address				IX	Output
		365	366	367	368		
		1	3	65	66	0	
200	1						
201							
202							
203	2						
204		2					
205						2	
206	65						
207							A
208							
200	2						
201							
202							
203	3						
204		3					
205						3	
206	66						
207							B
208							
200	3						
201							
202							
209							